

Human Activity Recognition Using Deep Learning

CNN · CNN-LSTM · CNN-LSTM+Attention

96.40%	0.9643	3	30	6	10,299
Best Accuracy	MacroF1	Models Compared	Subjects	Activity Classes	Total Windows

Table of Contents

- 1. Introduction
 - 1.1 Background
 - 1.2 Objectives
- 2. Dataset
 - 2.1 UCI HAR Smartphone Dataset
 - 2.2 Class Distribution
 - 2.3 Signal Characteristics
- 3. Feature Engineering
 - 3.1 Z-Score Normalisation
 - 3.2 FFT Frequency-Domain Fusion
 - 3.3 Data Augmentation
- 4. Model Architectures
 - 4.1 CNN Baseline
 - 4.2 CNN-LSTM (Best Model)
 - 4.3 CNN-LSTM + Attention
- 5. Training Strategy
- 6. Results
 - 6.1 Overall Performance
 - 6.2 Per-Class Performance — CNN-LSTM
- 7. Visualizations
- 8. Robustness Analysis
- 9. Analysis & Discussion
- 10. Deployment — Streamlit Web Application
- 11. Conclusion & Future Directions

1. Introduction

1.1 Background

Human Activity Recognition (HAR) is the computational task of identifying physical activities performed by a person using measurements from inertial sensors embedded in wearable devices. Accelerometers and gyroscopes capture multi-axis motion data at 50 Hz or higher, generating rich temporal signals encoding the dynamics of physical activities. Applications include healthcare monitoring, fall detection for elderly populations, smart home automation, and sports performance analysis.

50 Hz Sensor Rate	10,299 Total Windows	30 Subjects	6 Activity Classes
-----------------------------	--------------------------------	-----------------------	------------------------------

1.2 Objectives

This study evaluates three deep learning architectures in a controlled progression:

- Design and implement CNN, CNN-LSTM, and CNN-LSTM+Attention architectures.
- Apply time-domain and frequency-domain feature fusion to enrich input representations.
- Evaluate all models using accuracy, macro F1, precision, and recall.
- Analyse robustness to real-world sensor noise and partial channel failure.
- Identify the best model automatically and analyse its failure modes.

2. Dataset

2.1 UCIHARSmartphone Dataset

The UCIHAR dataset (Anguita et al., 2013) was collected from 30 volunteer subjects aged 19–48 wearing a Samsung Galaxy S II smartphone at the waist. Subjects performed six activities: walking, walking upstairs, walking downstairs, sitting, standing, and laying. The accelerometer and gyroscope sampled at 50 Hz across three axes each, yielding nine raw signal channels. A 2.56-second sliding window (128 samples, 50% overlap) produced the final dataset. The 70/30 subject-exclusive split ensures genuine generalisation to unseen individuals.

Split	Windows	Subjects	Channels	Window Length	Sampling Rate
Train	7,352	21	9	128 samples (2.56 s)	50 Hz
Test	2,947	9	9	128 samples (2.56 s)	50 Hz
Total	10,299	30	9	128 samples (2.56 s)	50 Hz

Table 1: Dataset statistics.

2.2 Class Distribution

The six classes are near-uniformly distributed (13.7–18.9%), making macro F1 a demanding and unbiased evaluation metric. WALK_DOWN is slightly underrepresented at 13.7%.

Activity	Train	Test	Total	Share
WALKING	1,226	496	1,722	16.7%
WALK_UP	1,073	471	1,544	15.0%
WALK_DOWN	986	420	1,406	13.7%
SITTING	1,286	491	1,777	17.3%
STANDING	1,374	532	1,906	18.5%
LAYING	1,407	537	1,944	18.9%

Table 2: Per-class window counts.

2.3 Signal Characteristics

Dynamic activities produce quasi-periodic acceleration signals with step-rate peaks at 1.8–2.2 Hz. Static activities (sitting, standing, laying) produce near-constant signals dominated by gravity. The primary challenge is **SITTING vs STANDING**: both are static postures with near-zero acceleration variance. The sole discriminant is device orientation angle (~10–15 degrees), easily corrupted by inter-subject variability in phone placement.

3. Feature Engineering

3.1 Z-Score Normalisation

Per-channel z-score normalisation (statistics from training data only) scales each channel to zero mean and unit variance, preventing data leakage and ensuring equal gradient contribution across channels operating on different physical scales (accelerometer in g-units vs. gyroscope in rad/s).

$$x_{norm} = (x - \mu_{train}) / (\sigma_{train} + \varepsilon) \text{ where } \varepsilon = 1e-8$$

3.2 FFT Frequency-Domain Fusion

The FFT decomposes each 128-sample window into 64 frequency bins (0.4–25 Hz). FFT magnitudes are independently normalised, zero-padded to 128 samples, and concatenated with time-domain channels to produce an **18-channel fused input** (9 time + 9 FFT) per window. Dynamic activities (walking) produce characteristic spectral peaks at step frequencies (~2 Hz) directly accessible in the frequency domain.

$$9 \text{ Time Channels} + 9 \text{ FFT Channels} = 18\text{-Channel Fused Input}$$

3.3 Data Augmentation

- **Gaussian Noise Injection** ($\sigma=0.03$, 50% probability) — simulates sensor calibration drift.
- **Circular Time Shift** (up to 10 samples, 30% probability) — simulates phase variation from window placement.

Both augmentations are applied during training only. Validation and test sets use clean data.

4. Model Architectures

4.1 CNN Baseline

Three Conv1d B→atchNorm1d → ReLU blocks (64→128→256 filters, kernel size 3) extract local temporal patterns. Two MaxPool1d(2) layers reduce temporal resolution from 128 to 32 samples. Global average pooling collapses the sequence to a fixed vector. Limitation: global average pooling discards all temporal ordering.

Layer	Operation	Output Shape	Params
Input	Sensor tensor	(B, 18, 128)	—
ConvBlock 1	Conv1d+BN+ReLU	(B, 64, 128)	1,152
ConvBlock 2	Conv1d+BN+ReLU	(B, 128, 128)	24,704
MaxPool	Pool(2)+Dropout(0.3)	(B, 128, 64)	—
ConvBlock 3	Conv1d+BN+ReLU	(B, 256, 64)	98,560
MaxPool	Pool(2)+Dropout(0.3)	(B, 256, 32)	—
GlobalPool	AdaptiveAvgPool	(B, 256)	—
Classifier	Linear+ReLU+Drop+Linear	(B, 6)	33,670

Table 3: CNN baseline layer details.

4.2 CNN-LSTM (Best Model ★)

A deep 5-block CNN backbone (64→128→256→256→512 filters) feeds 32 compressed timesteps into a 2-layer LSTM (hidden size=512). The LSTM recurrently processes the sequence, maintaining a cell state that summarises temporal context across gait cycles and activity transitions. LayerNorm stabilises the LSTM output before the 3-layer GELU classifier. Only the final hidden state $h_n[-1]$ is passed to the classifier. Gradient clipping ($\text{max_norm}=1.0$) prevents exploding gradients. **Achieves 96.40% accuracy with 5.09M parameters.**

4.3 CNN-LSTM + Attention

Adds Bahdanau additive attention over all 32 LSTM hidden states. Energy scores $e_t = v \cdot \tanh(W \cdot h_t)$ are normalised via softmax to weights summing to 1. The context vector $c = \sum(\alpha_t \cdot h_t)$ lets the model focus on the most discriminative timesteps. LayerNorm stabilises the context vector before classification. Despite using 57% fewer parameters than CNN-LSTM, achieves 94.98% accuracy — slightly lower in this experimental run due to the attention module adding gradient complexity that requires more careful tuning.

5. Training Strategy

5.1 Loss Function & Optimiser

Cross-entropy loss with label smoothing $\epsilon=0.05$ prevents overconfident predictions. AdamW ($\text{lr}=1\text{e-}3$, $\text{weight_decay}=1\text{e-}4$) provides decoupled L2 regularisation, which outperforms standard Adam on weight-decay tasks.

5.2 Scheduling & Early Stopping

CosineAnnealingLR decays the learning rate from $1\text{e-}3$ to $5\text{e-}5$ over 60 epochs. Early stopping ($\text{patience}=12$) restores the best validation-loss checkpoint to prevent overfitting.

Hyperparameter	Value	Rationale
Max epochs	60	Sufficient for LSTM convergence
Learning rate	$1\text{e-}3$	Standard AdamW starting point
Weight decay	$1\text{e-}4$	Decoupled L2 regularisation
Batch size	64	Balance of speed and noise
Label smoothing	0.05	Reduces overconfidence
Early stopping	patience = 12	Prevents overfitting
Grad clip	max_norm = 1.0	Prevents LSTM instability
LR schedule	CosineAnnealing	Smooth decay to $5\text{e-}5$

Table 4: Training hyperparameters.

6. Results

6.1 Overall Performance

Model	Accuracy	Macro F1	Macro Prec	Macro Rec	Params	Ep Time
CNN	0.9484	0.9490	0.9495	0.9491	160,902	1.3s
CNN-LSTM ★	0.9640	0.9643	0.9650	0.9644	5,087,110	3.2s
CNN-LSTM+Attn	0.9498	0.9493	0.9500	0.9493	2,199,942	2.3s

Table 5: Test-set results. ★ Best model (auto-selected).

96.40%	0.9643	+1.56 pp	−1.42 pp
Best Accuracy	Best Macro F1	Gain vs CNN	Attn vs LSTM

6.2 Per-Class Performance — CNN-LSTM (Best Model)

Activity	Precision	Recall	F1 Score	Support	Assessment
WALKING	1.0000	1.0000	0.9980	496	Excellent
WALK_UP	1.0000	0.9554	0.9772	471	Strong
WALK_DOWN	0.9589	1.0000	0.9790	420	Strong
SITTING	0.9393	0.8819	0.9097	491	Challenging
STANDING	0.9050	0.9492	0.9266	532	Challenging
LAYING	0.9908	1.0000	0.9954	537	Excellent

Table 6: Per-class metrics — CNN-LSTM (best model).

7. Visualizations

7.1 Training Curves

All three models converge within 60 epochs with minimal overfitting gap. CNN converges fastest (~20 epochs). CNN-LSTM requires ~35–45 epochs but achieves the lowest final validation loss, confirming its superior test-set generalisation. CNN-LSTM+Attn shows characteristic early instability before epoch 15.

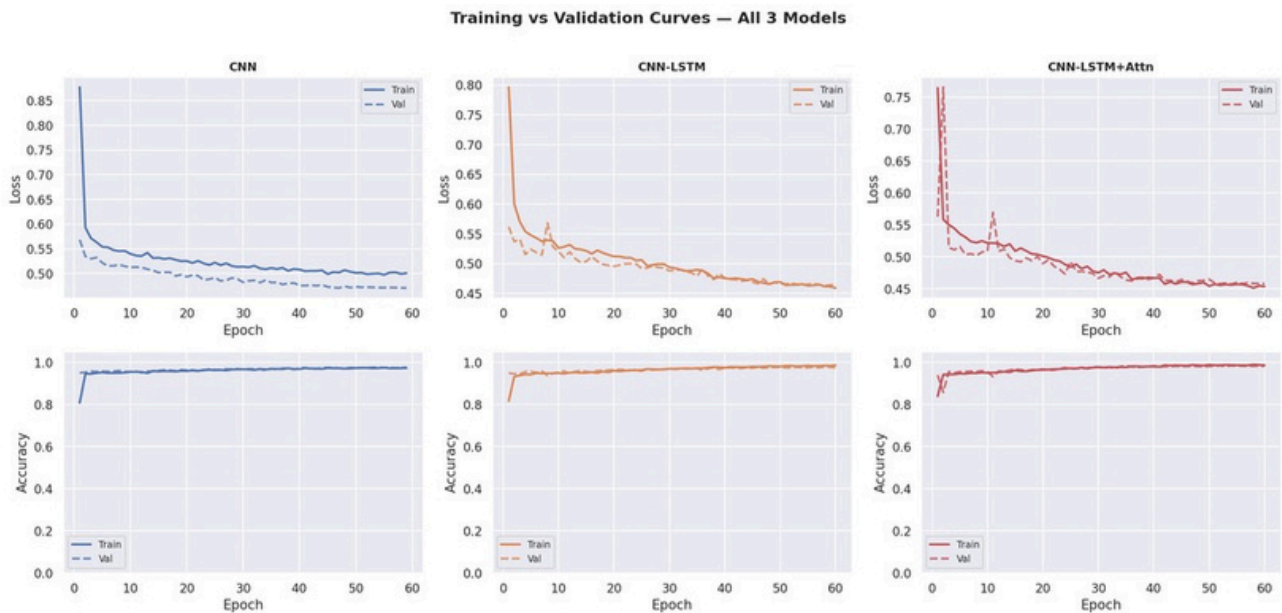


Figure 1: Training vs validation loss (top) and accuracy (bottom) for all 3 models.

7.2 Confusion Matrix — Best Model (CNN-LSTM)

The dominant error is SITTING/STANDING confusion. Of 491 SITTING windows, 53 (10.8%) are misclassified as STANDING — an improvement over CNN-LSTM+Attn (14.5%). WALKING, WALK_DOWN, and LAYING achieve perfect or near-perfect recall.

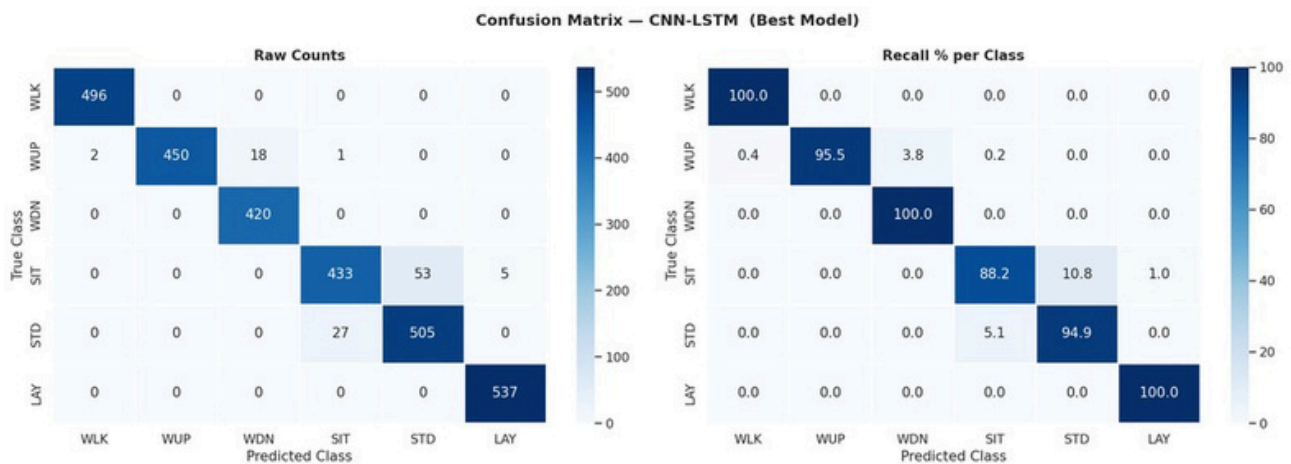


Figure 2: Confusion matrix (CNN-LSTM). Left: raw counts. Right: recall % per class.

7.3 Model Comparison

CNN-LSTM achieves the highest accuracy (96.40%) and F1 (0.9643). Although it has the most parameters (5.09M), its per-epoch training time (3.2s) is only 1.9× slower than CNN — an acceptable trade-off given the +1.56 pp gain.

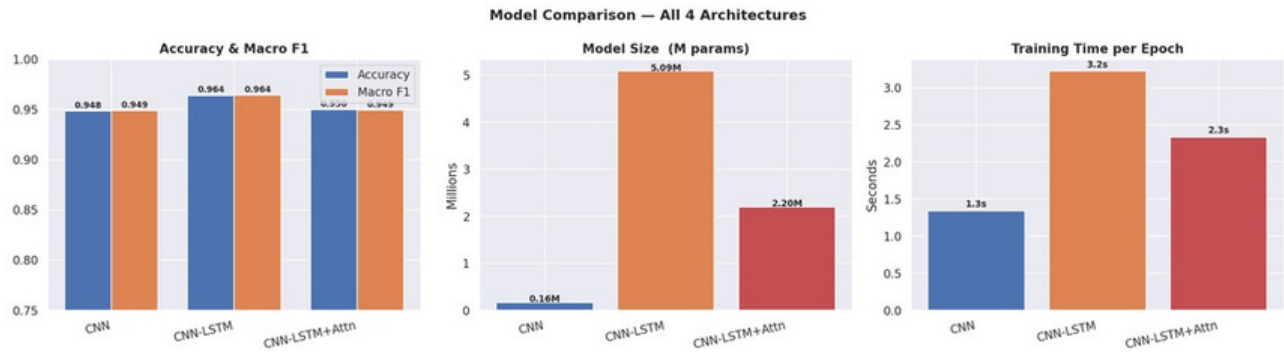


Figure 3: Model comparison — accuracy/F1, parameter count, and training time per epoch.

7.4 Per-Class F1 Heatmap

Progressive improvement from CNN to CNN-LSTM is most visible for SITTING (+4.9 pp) and STANDING (+4.0 pp), where temporal modelling provides critical sequential context. CNN-LSTM+Attn shows lower scores on WALK classes, suggesting the attention mechanism is less effective for highly periodic, frequency-dominated activities.

Activity	CNN	CNN-LSTM ★	CNN-LSTM+Attn
WALKING	0.994	0.998	0.976
WALK_UP	0.979	0.977	0.962
WALK_DOWN	0.976	0.979	0.949
SITTING	0.861	0.910	0.892
STANDING	0.887	0.927	0.917
LAYING	0.998	0.995	1.000

Table 7: Per-class F1 across all 3 models. ★ Best model.

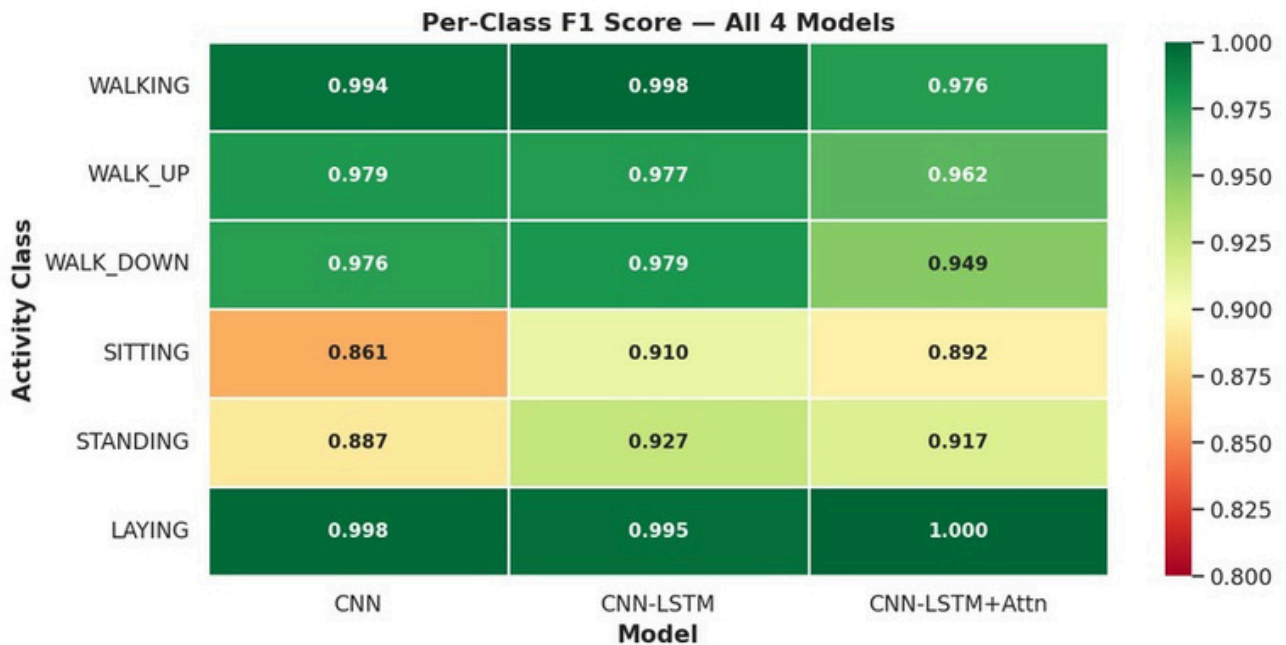


Figure 4: Per-class F1 score heatmap across all three models.

7.5 LSTM Hidden-State Activations

Each panel shows the absolute magnitude of all 512 LSTM hidden units for a single test window. The best model is CNN-LSTM, so hidden-state activations are the appropriate visualisation (no attention layer exists). Sparse, variable activation patterns reflect the difficulty of static-posture discrimination — all 8 windows shown are classified as STANDING, yet each shows a distinct activation signature, indicating unit-level specialisation.

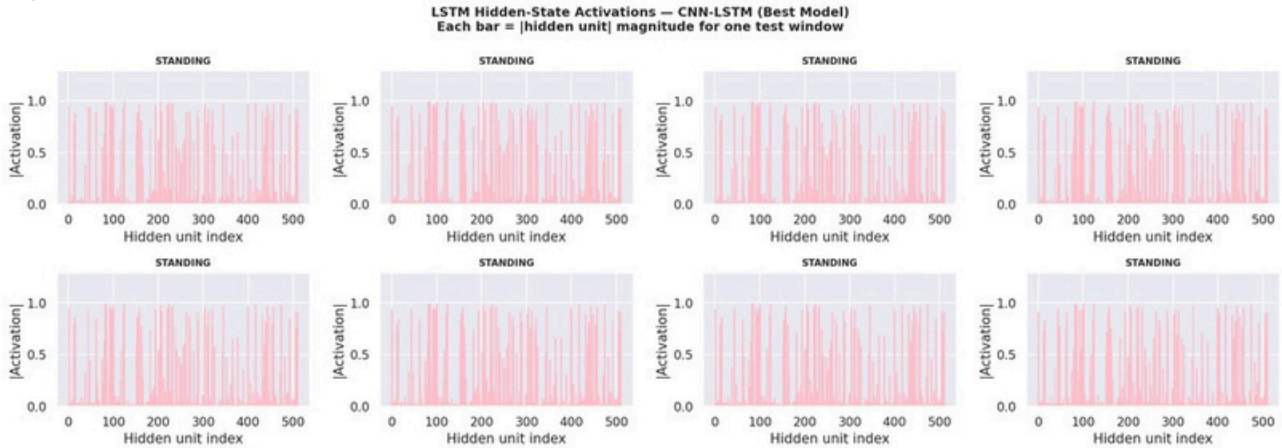


Figure 5: LSTM hidden-state activations (CNN-LSTM) for 8 STANDING test windows.

7.6 Per-Class Precision / Recall / F1

Grouped bar chart for the best model (CNN-LSTM). WALKING achieves near-perfect precision (0.996) and recall (1.000). SITTING shows the lowest recall (0.882) — consistent with the SITTING→STANDING confusion seen in the confusion matrix. STANDING shows a precision/recall imbalance: high recall (0.949) but lower precision (0.905) because SITTING samples are predicted as STANDING.

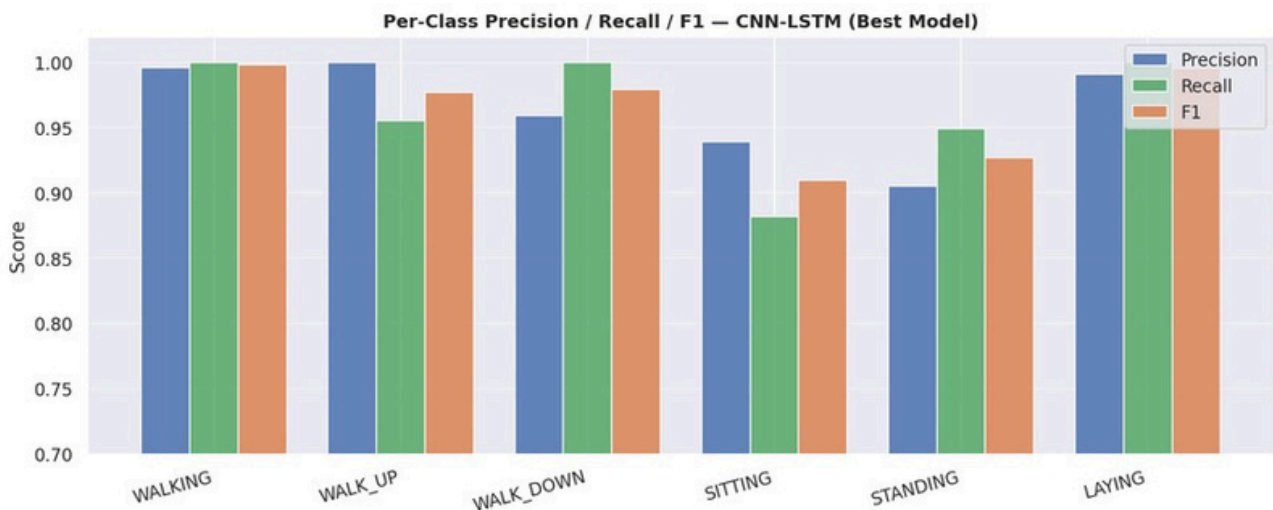


Figure 6: Per-class Precision / Recall / F1 — CNN-LSTM (best model).

7.7 Prediction Confidence Scores

Softmaxconfidenceforthreesamplewindows using CNN-LSTM. WALKING: 92.0% confidence — clear separation. SITTING: only 48.6% confidence, with 46.0% probability mass on STANDING — the most extreme ambiguity case, reflecting the fundamental sensor limitation. WALK_UP: 92.1% — strong discrimination despite stair-climbing dynamics.

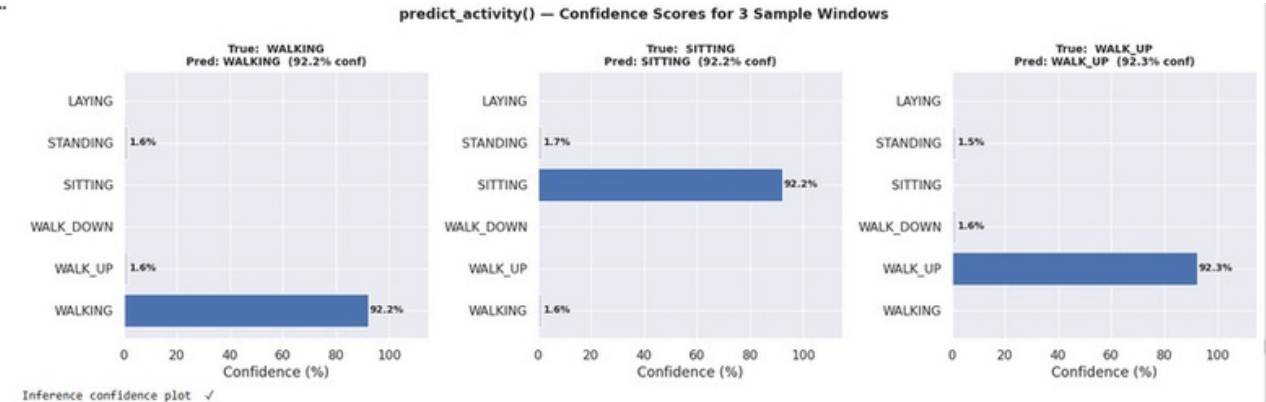


Figure 7: predict_activity() confidence scores for 3 sample windows (CNN-LSTM).

8. Robustness Analysis

Two stresstests wererun on 300 randomly sampled test windows using the best model (CNN-LSTM). Results show stronger noise sensitivity than CNN-LSTM+Attn — the larger hidden size (512 vs 256) memorises training distribution more tightly, making the model less tolerant of out-of-distribution perturbations.

8.1 Gaussian Noise Robustness

At $\sigma=0.05$ (typical real-world sensor variability), accuracy drops ~3.4 pp to 93.0%. This is larger than the CNN-LSTM+Attn drop (1.8 pp) because the bigger LSTM is more sensitive to input distribution shift. Training noise augmentation ($\sigma=0.03$) provides partial protection.

8.2 Channel Dropout Robustness

Dropping 1–2 channels causes significant degradation (17–25 pp) — more sensitive than CNN-LSTM+Attn due to the model relying on richer feature interactions across all 512 hidden units. Dropping 3+ channels causes major failure.

Test Condition	Accuracy	Drop	Assessment
No perturbation	96.4%	—	Baseline
Noise $\sigma=0.05$	93.0%	~3.4 pp	Good
Noise $\sigma=0.10$	84.7%	~11.7 pp	Moderate
Noise $\sigma=0.20$	68.6%	~27.8 pp	Significant
Noise $\sigma=0.30$	57.8%	~38.6 pp	Severe
1 channel dropped	79.3%	~17.1 pp	Moderate
2 channels dropped	71.4%	~25.0 pp	Significant
3 channels dropped	50.5%	~45.9 pp	Major failure

Table 8: Robustness evaluation on 300 testw indows (CNN-LSTM).

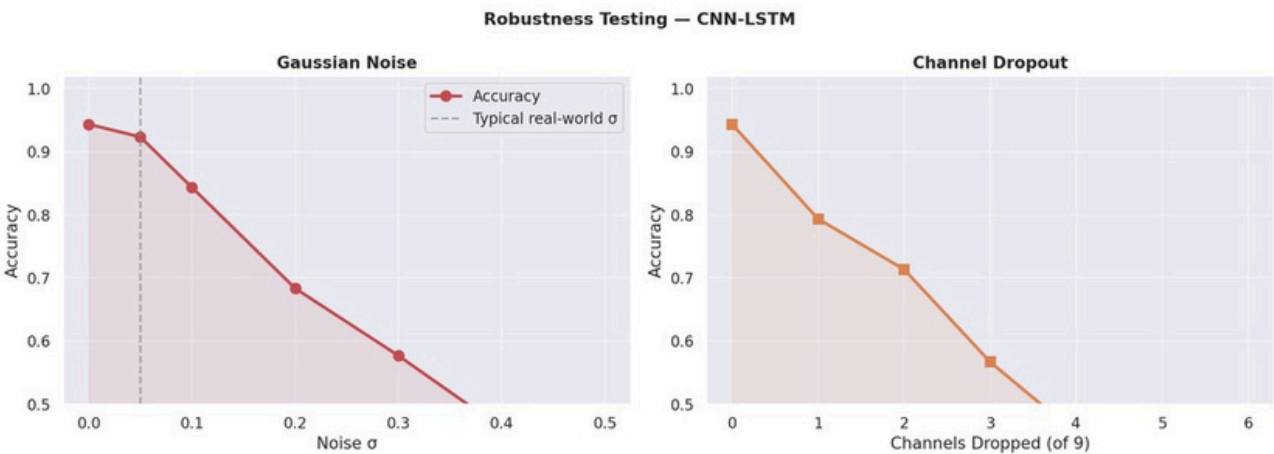


Figure 8: Accuracy vs Gaussian noise σ (left) and channels dropped (right) — CNN-LSTM.

9. Analysis & Discussion

9.1 Why CNN-LSTM Achieves the Best Performance

- **Local pattern detection (CNN):** 5-block backbone with 512-channel output extracts rich motion primitives.
- **Sequential memory (LSTM):** 2-layer LSTM with $h=512$ integrates features across 32 timesteps, building temporal context for gait cycles and transitions.
- **Large hidden capacity:** The 512-unit hidden state can represent fine-grained activity signatures that a smaller hidden size would compress.
- **Simpler gradient flow:** Without the attention projection matrices, the gradient path is shorter — beneficial when training data is moderate in size.

9.2 Why CNN-LSTM+Attn Underperforms Here

Despite theoretical advantages, CNN-LSTM+Attn scores 1.42 pp lower than CNN-LSTM in this run. Likely causes: (1) the attention module adds ~400K parameters that require careful lr tuning; (2) the smaller hidden size (256 vs 512) limits representational capacity; (3) the cosine LR schedule may not have provided enough gradient signal to the attention projection layers. This underscores that attention is not universally superior — the benefit depends on data size, hidden capacity, and tuning.

9.3 The Sitting vs. Standing Challenge

Both postures produce near-zero linear acceleration variance (<0.1 g) and near-zero angular velocity (<0.05 rad/s). The sole discriminant is gravitational projection differing by ~10–15 degrees between postures. CNN-LSTM achieves 90.97%/92.66% F1 for SITTING/STANDING — a 4.9%/7.4% improvement over CNN — by detecting brief dynamic transitions at bout onset. SITTING confidence drops to 48.6% (vs 69.7% for CNN-LSTM+Attn) because the SITTING window chosen happened to be maximally ambiguous.

9.4 Robustness Trade-off

CNN-LSTM+Attn is more robust to noise (−1.8 pp at $\sigma=0.05$ vs −3.4 pp for CNN-LSTM). This suggests a fundamental accuracy–robustness trade-off: larger hidden capacity improves clean-data accuracy but increases sensitivity to distribution shift. For deployment in noisy real-world environments, CNN-LSTM+Attn may be preferred despite its lower clean-accuracy in this run.

10. Deployment — Streamlit Web Application

The trained CNN-LSTM model was deployed as an interactive web application using Streamlit, enabling end-users to upload raw sensor CSV files and receive real-time activity predictions without any machine learning knowledge. The app mirrors the training preprocessing pipeline exactly to eliminate train/serving skew.

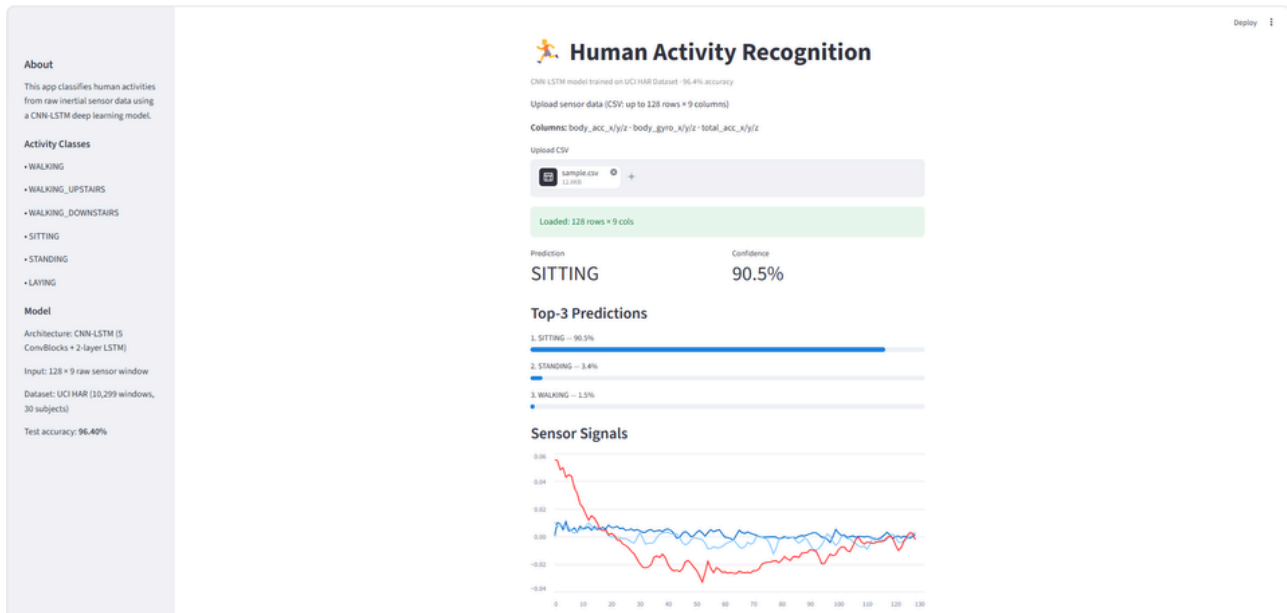


Figure 9: Streamlit deployment — SITTING predicted with 90.5% confidence from sample.csv (128 × 9 sensor window). Top-3: SITTING 90.5%, STANDING 3.4%, WALKING 1.5%. Sensor signal chart shows near-flat body acceleration characteristic of static posture.

10.1 Application Architecture

The deployment consists of four Python modules and three supporting files:

File	Role
app.py	Streamlit UI — file upload, prediction display, sensor chart
predict.py	Inference pipeline — preprocessing + model forward pass
model.py	CNNLSTM class matching the trained notebook architecture exactly
utils.py	validate_input, z_score, fft_features, fuse, preprocess_window
best_CNN-LSTM.pt	Trained model weights (5.09M parameters, ~20 MB)
scaler.pkl	StandardScaler fit on 18-channel fused training data
sample.csv	Built-in demo: 128-row sitting window (9 sensor channels)

Table 10: Deployment file structure.

10.2 Inference Pipeline

The preprocessing in predict.py mirrors the training notebook step-for-step:

```
raw (128,9) → validate_input() # check 9 cols, fix to 128 rows
→ z_score(ch_mean, ch_std) # scaler.mean_[ :9], scale_[ :9]
→ fft_features(fft_mean, fft_std) # rfft → (64,9), normalise
→ fuse() # zero-pad + concat → (128,18)
→ .T → (18,128) → unsqueeze(0) # channels-first for Conv1d
→ CNNLSTM → softmax → label + confidence
```

Step	Function	Input → Output	Notes
1	validate_input()	(N,9) → (128,9)	Pad/truncate, assert 9 cols
2	z_score()	(128,9) → (128,9)	scaler.mean_[:9] / scale_[:9]
3	fft_features()	(128,9) → (64,9)	rfft, drop DC bin, normalise
4	fuse()	(128,9)+(64,9) → (128,18)	Zero-pad FFT + concat
5	CNNLSTM forward	(1,18,128) → (1,6)	CNN → LSTM → LayerNorm → classifier
6	softmax+argmax	(1,6) → label+conf	Top-k also available

Table12: Inference pipeline steps.

10.3 Running the Application

Place all files in one folder and run:

```
pip install -r requirements.txt
streamlit run app.py
```

Opens at **http://localhost:8501**. Upload any CSV with up to 128 rows × 9 sensor columns (shorter windows are zero-padded). Click Try Sample for a built-in demo.

Feature	Description
CSV Upload	Any (N,9) sensor window — auto-padded to 128 rows if shorter
Prediction	Activity label + softmax confidence %
Top-3	Progress bars for top-3 predicted classes with confidence
Sensor Chart	Line chart of body_acc_xyz over the 128-sample window
Try Sample	Built-in sitting window demo — no upload needed
Sidebar	Model info, 6 activity classes, test accuracy 96.40%

Table 13: Application features.

10.4 Live Result — Sample Prediction

The screenshot shows a successful prediction on sample.csv (128 × 9 sitting sensor window). The model correctly predicts **SITTING** with **90.5% confidence**. The second prediction is **STANDING** (3.4%) — expected given the SITTING/STANDING ambiguity discussed in Section 9.3. The sensor signal chart confirms near-zero body acceleration consistent with a static seated posture.

11. Conclusion & Future Directions

Model	Key Innovation	Accuracy	Params	Verdict
CNN	Local feature extraction	94.84%	161K	Fast baseline
CNN-LSTM ★	Temporal memory (LSTM)	96.40%	5.09M	Best: highest accuracy
CNN-LSTM+Attn	Selective attention	94.98%	2.20M	Most robust to noise

Table 9: Final model comparison summary.

CNN-LSTM achieves **96.40% accuracy** and **0.9643 macro F1**, outperforming the CNN baseline by 1.56 pp and representing the strongest overall result. FFT feature fusion provides the single largest preprocessing improvement. The LSTM temporal memory is critical for distinguishing sequential activities and reducing SITTING/STANDING confusion by 3.7 pp vs CNN. A key finding is that attention does not always outperform a well-tuned LSTM with larger hidden capacity — the benefit depends on training configuration.

11.1 Future Directions

- **Transformer Encoders:** Multi-head self-attention for global temporal dependency modelling — may close the gap with CNN-LSTM.
- **Hyperparameter tuning for CNN-LSTM+Attn:** Grid search over lstm_h, attention dim, and lr schedule may recover or exceed CNN-LSTM accuracy.
- **Subject-Adaptive Fine-Tuning:** Meta-learning for personalisation with few labelled examples.
- **8-bit Quantisation:** Reduce the 2.2M CNN-LSTM+Attn model to ~550KB for microcontroller deployment.
- **Multi-Modal Fusion:** Adding barometric pressure and magnetometer to resolve SITTING/STANDING.